Banco de 4 exámenes tipo test

Redes neuronales con PyTorch

Cada examen: 25 preguntas (Parte I teoría y Parte II análisis de código) y una única respuesta correcta por pregunta. Solucionario con explicaciones al final.

Examen	Contenido principal
Examen 1	Clasificación multiclase con MLP, Dropout y Adam (dataset Wine)
Examen 2	Regresión con MLP, SGD con momentum y early stopping (viviendas)
Examen 3	Redes convolucionales (CNN) sobre MNIST
Examen 4	Clasificación binaria con BCEWithLogits, scheduler y evaluación (Breast Cancer)
Solucionario	Claves de respuesta y explicación de cada pregunta

Consejos: en la Parte II asume que el código se ejecuta sin errores salvo que la pregunta indique una modificación. Intenta resolver cada examen en unos 50 minutos sin mirar las respuestas.

EXAMEN 1

Clasificación multiclase con MLP, Dropout y Adam (dataset Wine)
Nombre: _____ Nota: _____ / 25

PARTE I - Teoría (preguntas 1 a 14)

1. Una red de clasificación con 4 clases produce los logits (1, 1, 1, 1) para una muestra. ¿Qué probabilidades devuelve Softmax?

A) 1.0 para la primera clase y 0 para las demás.
B) 0.5 para cada una de las cuatro clases.
C) e (aprox. 2.72) para cada clase, sin normalizar.
D) 0.25 para cada una de las cuatro clases.
E) No está definido cuando todos los logits son iguales.

2. ¿Qué describe el problema de la «dying ReLU»?

A) La ReLU se satura en 1 para entradas muy grandes y su gradiente se vuelve casi nulo.
B) La ReLU devuelve valores negativos que hacen que la loss sea NaN en todas las epochs.
C) Las neuronas con ReLU se desactivan al azar en cada paso, igual que con Dropout.
D) El gradiente de la ReLU crece sin control cuando la entrada es positiva y grande.
E) Neuronas que siempre reciben entradas negativas dan salida 0, su gradiente es 0 y dejan de aprender.

3. Una imagen puede contener a la vez un gato, un perro y/o un pájaro (etiquetas independientes, no excluyentes). ¿Qué salida y coste son adecuados?

A) 3 neuronas con Sigmoid y Binary Cross-Entropy aplicada a cada salida.
B) 3 neuronas con Softmax y Multiclass Cross-Entropy sobre las tres salidas.
C) 1 neurona sin activación y MSE sobre el número de objetos presentes.
D) 3 neuronas con ReLU y Multiclass Cross-Entropy sobre las tres salidas.
E) 1 neurona con Sigmoid y Binary Cross-Entropy sobre la clase más frecuente.

4. Una red recibe 10 variables, tiene una capa oculta de 6 neuronas y una capa de salida de 2 neuronas (todas con bias). ¿Cuántos parámetros entrenables tiene?

A) 72
B) 60
C) 80
D) 92
E) 78

5. Para una muestra, la red asigna probabilidad 0.5 a la clase correcta. ¿Cuánto vale la Cross-Entropy de esa muestra?

A) 0.5
B) 1.0
C) 2.0
D) ln 2, aprox. 0.69
E) 0, porque la predicción es correcta

6. Durante el entrenamiento la loss da saltos enormes y acaba en NaN. ¿Cuál es la causa más probable?

A) Learning rate demasiado pequeño, que bloquea los gradientes.
B) Learning rate demasiado grande, que hace divergir la optimización.
C) Batch size demasiado pequeño en el conjunto de validación.
D) Early stopping con una paciencia demasiado alta.
E) Haber estandarizado las variables de entrada a media 0.

7. Un dataset de 1000 muestras se entrena con batch_size = 50 durante 4 epochs. ¿Cuántas actualizaciones de parámetros se hacen en total?

A) 20
B) 200
C) 80
D) 50
E) 4000

8. ¿Qué hace una capa de Batch Normalization durante el entrenamiento?

A) Elimina del batch las muestras cuya loss supera un umbral fijado.
B) Divide el learning rate por el tamaño del batch en cada paso.
C) Desactiva al azar una fracción de las neuronas de la capa.
D) Convierte las etiquetas del batch en vectores one-hot.
E) Normaliza las activaciones con la media y varianza del mini-batch y aplica una escala y un desplazamiento aprendidos.

9. Un modelo obtiene un 60 % de accuracy en train y un 58 % en validación. ¿Cuál es el diagnóstico más probable?

A) Overfitting: la red ha memorizado el conjunto de entrenamiento.
B) Data leakage entre el conjunto de train y el de validación.
C) Exploding gradients durante las primeras epochs.
D) El modelo está bien ajustado y no necesita cambios.
E) Underfitting: el modelo no tiene capacidad o entrenamiento suficientes.

10. ¿Qué caracteriza al optimizador Adam?

A) Usa un learning rate fijo e idéntico para todos los parámetros.
B) Combina momentum con un learning rate adaptativo por parámetro basado en gradientes al cuadrado.
C) Calcula el gradiente exacto sobre todo el dataset en cada paso.
D) Elimina el término de bias de todas las capas lineales.
E) Sustituye la backpropagation por una búsqueda aleatoria de pesos.

11. ¿Qué efecto suele tener la regularización L1 sobre los pesos de la red?

A) Tiende a llevar muchos pesos exactamente a cero, produciendo modelos dispersos.
B) Aumenta el valor absoluto de los pesos más pequeños.
C) Solo modifica los bias y deja los pesos intactos.
D) Escala los pesos para que su suma sea siempre igual a 1.
E) Impide que ningún peso de la primera capa sea distinto de cero.

12. Estandarizas las variables con la media y desviación de todo el dataset y después divides en train/val/test. ¿Cuál es el problema?

A) Fuga de información: estadísticos de val/test influyen en el preprocesado del entrenamiento.
B) Ninguno: es la forma recomendada de estandarizar los datos.
C) Las variables quedan con varianza distinta de 1 en el conjunto de train.
D) El modelo no podrá usar ReLU con datos estandarizados.
E) Se pierde el orden de las muestras y ya no se puede barajar.

13. ¿Cuál de las siguientes medidas NO ayuda normalmente a reducir el overfitting?

A) Aplicar Dropout en las capas ocultas.
B) Usar early stopping sobre la validation loss.
C) Añadir weight decay (regularización L2).
D) Añadir más capas y neuronas para aumentar la capacidad del modelo.
E) Conseguir más datos de entrenamiento o aplicar data augmentation.

14. Un dataset tiene un 95 % de muestras de la clase A y un 5 % de la clase B. Un clasificador predice siempre A. ¿Qué conclusión es correcta?

A) Es un excelente modelo, porque su accuracy es del 95 %.
B) Tiene un 50 % de accuracy, porque solo acierta una de las dos clases.
C) Su recall en B es 1, porque nunca se equivoca al clasificar una muestra de A.
D) Tiene un 95 % de accuracy pero recall 0 en B: la accuracy engaña con clases desbalanceadas.
E) Está sufriendo un overfitting severo a la clase B.

PARTE II - Análisis de código (preguntas 15 a 25)

Responde a las preguntas 15 a 25 usando el siguiente script de PyTorch. Se asume que el código se ejecuta sin errores salvo que una pregunta indique una modificación.

```
1  import torch
2  from torch import nn
3  from torch.utils.data import TensorDataset, DataLoader
4  from sklearn import datasets
5
6  wine = datasets.load_wine()   # 178 muestras, 13 variables, 3 clases
```

```
 7  X = torch.tensor(wine.data, dtype=torch.float)
 8  y = torch.tensor(wine.target, dtype=torch.long)
 9
10  idx = torch.randperm(178)
11  X_train, y_train = X[idx[:128]], y[idx[:128]]
12  X_val, y_val = X[idx[128:]], y[idx[128:]]
13
14  model = nn.Sequential(
15      nn.Linear(13, 16),
16      nn.ReLU(),
17      nn.Dropout(p=0.25),
18      nn.Linear(16, 8),
19      nn.ReLU(),
20      nn.Linear(8, 3),
21  )
22
23  criterion = nn.CrossEntropyLoss()
24  optim = torch.optim.Adam(model.parameters(), lr=0.01)
25
26  train_loader = DataLoader(TensorDataset(X_train, y_train), batch_size=32, shuffle=True)
27  val_loader = DataLoader(TensorDataset(X_val, y_val), batch_size=32, shuffle=False)
28
29  epochs = 20
30  for epoch in range(epochs):
31      model.train()
32      train_loss = 0.
33      for x, t in train_loader:
34          optim.zero_grad()
35          out = model(x)
36          loss = criterion(out, t)
37          loss.backward()
38          optim.step()
39          train_loss += loss.item() * x.size(0)
40      train_loss /= len(X_train)
41
42      model.eval()
43      val_loss = 0.
44      correct = 0
45      with torch.no_grad():
46          for x, t in val_loader:
47              out = model(x)
48              val_loss += criterion(out, t).item() * x.size(0)
49              correct += (out.argmax(dim=1) == t).sum().item()
50      val_loss /= len(X_val)
51      val_acc = correct / len(X_val)
```

15. ¿Cuántos parámetros entrenables tiene `model` (líneas 14-21)?

A) 360
B) 399
C) 224
D) 411
E) 387

16. Se cambia la línea 18 por `nn.Linear(16, 12)` y la línea 20 por `nn.Linear(12, 3)`. ¿Cuántos parámetros entrenables tiene ahora la red?

A) 436
B) 467
C) 455
D) 491
E) 387

17. ¿Cuántas veces se ejecuta `optim.step()` en todo el entrenamiento?

A) 20
B) 80
C) 4
D) 128
E) 2560

18. ¿Cuál es la forma (shape) de `out` en cada iteración del bucle de entrenamiento (línea 35)?

A) (128, 3)
B) (3, 32)
C) (32,)
D) (32, 3)
E) (32, 13)

19. El `val_loader` tiene 50 muestras y `batch_size=32`. ¿Cuántos batches recorre la validación y de qué tamaño?

A) 2 batches de 25 muestras cada uno.
B) 1 batch con las 50 muestras.
C) 2 batches: uno de 32 muestras y otro de 18.
D) 50 batches de una muestra cada uno.
E) 2 batches: uno de 32 muestras y otro de 20.

20. ¿Qué hace `model.eval()` (línea 42) sobre la capa Dropout?

A) La mantiene activa pero con p = 0.5 en validación.
B) La elimina de forma permanente del modelo.
C) La desactiva: deja pasar todas las activaciones sin ponerlas a cero.
D) Pone a cero los gradientes de las capas siguientes.
E) Congela los pesos de la capa Linear anterior.

21. ¿Por qué se multiplica `loss.item()` por `x.size(0)` en la línea 39?

A) Para escalar el gradiente según el número de muestras del batch.
B) Para ponderar cada batch por su tamaño y obtener la media exacta por muestra.
C) Porque `criterion` devuelve la suma de las pérdidas y no su media.
D) Para que la loss resulte siempre menor que 1 al final de cada epoch.
E) Para compensar el efecto del Dropout sobre el tamaño del batch.

22. Se elimina `with torch.no_grad():` (línea 45) de la validación. ¿Qué ocurre?

A) Todo funciona igual, pero se gasta más memoria y tiempo al construir el grafo de gradientes.
B) Los pesos se actualizan con los datos de validación.
C) El código lanza un error porque falta el cálculo de gradientes.
D) La val_loss pasa a valer siempre cero.
E) Dropout se vuelve a activar durante la validación.

23. Se mueve `optim.zero_grad()` de la línea 34 a justo después de `loss.backward()` (antes de `optim.step()`). ¿Qué ocurre?

A) Nada relevante: PyTorch reinicia los gradientes por su cuenta.
B) Los gradientes se acumulan y la red aprende más rápido.
C) Se borran los gradientes antes del paso, así que los pesos no se actualizan y la red no aprende.
D) El código lanza un error porque falta el grafo de cómputo.
E) Solo se actualizan los bias y no los pesos.

24. La línea 36 se reescribe como `loss = criterion(out.softmax(dim=1), t)`. ¿Qué ocurre?

A) Es la práctica recomendada: así la loss recibe probabilidades.
B) Lanza un error porque Softmax no admite `dim=1`.
C) Es equivalente al código original, porque Softmax es idempotente.
D) Solo cambia el signo de la loss final.
E) Se ejecuta, pero es incorrecto: CrossEntropyLoss ya incluye log-softmax y se aplicaría dos veces.

25. Al final de una epoch, `val_acc` vale 0.92. ¿Cuántas muestras de validación se han clasificado bien?

A) 46
B) 92
C) 23
D) 48
E) 4

EXAMEN 2

Regresión con MLP, SGD con momentum y early stopping (viviendas)
Nombre: _____ Nota: _____ / 25

PARTE I - Teoría (preguntas 1 a 14)

1. Una red predice (2, 4) para dos muestras cuyos valores reales son (3, 1). ¿Cuánto vale el MSE?

A) 2
B) 10
C) 2.24
D) 5
E) 4

2. ¿Qué describe el problema del «vanishing gradient»?

A) Los gradientes se hacen muy pequeños al propagarse hacia las primeras capas, que apenas aprenden.
B) Los gradientes crecen exponencialmente y los pesos acaban valiendo NaN.
C) El gradiente se anula únicamente cuando la loss de train llega a cero.
D) Los gradientes desaparecen porque el batch size es demasiado grande.
E) Las neuronas de la última capa dejan de recibir información de entrada.

3. ¿Por qué NO es buena idea inicializar todos los pesos de una capa oculta a cero?

A) Porque PyTorch solo admite inicializaciones con valores positivos.
B) Porque los gradientes serían infinitos en el primer paso.
C) Porque la loss no podría calcularse con pesos nulos.
D) Porque se desactivaría el bias de todas las capas de la red.
E) Todas sus neuronas recibirían el mismo gradiente y seguirían siendo idénticas (simetría).

4. ¿Qué rango de salida tiene la activación tanh?

A) Entre -1 y 1, y está centrada en cero.
B) Entre 0 y 1, igual que la sigmoide.
C) Entre 0 e infinito, igual que la ReLU.
D) Entre -infinito e infinito, sin saturación.
E) Solo toma los valores -1, 0 y 1.

5. Una red recibe 4 variables y tiene capas de 5, 3 y 1 neuronas (todas con bias). ¿Cuántos parámetros entrenables tiene?

A) 47
B) 38
C) 44
D) 57
E) 41

6. Un dataset tiene 1050 muestras y se usa `batch_size=100` con `drop_last=False` (por defecto). ¿Cuántos batches hay por epoch?

A) 10
B) 12
C) 105
D) 1050
E) 11

7. ¿Por qué se suelen estandarizar las variables de entrada antes de entrenar una red?

A) Porque PyTorch no acepta variables con valores superiores a 1.
B) Escalas muy distintas deforman la superficie de la loss y ralentizan o desestabilizan el descenso.
C) Para que ya no haga falta un conjunto de validación.
D) Para convertir las variables numéricas en categóricas.
E) Porque así el modelo no necesita bias en ninguna capa.

8. ¿Qué ventaja tiene entrenar con mini-batches frente a usar todo el dataset en cada actualización?

A) Garantiza que cada paso siga exactamente la dirección de máxima bajada de la loss global.
B) Actualiza los pesos con más frecuencia y usa menos memoria; el ruido del gradiente puede ayudar a escapar de mínimos malos.
C) Elimina la necesidad de elegir un learning rate.
D) Hace el entrenamiento determinista y perfectamente reproducible.
E) Evita el overfitting en cualquier circunstancia.

9. La train loss baja en todas las epochs; la val loss baja hasta la epoch 15 y sube a partir de ahí. ¿Qué indica y qué conviene hacer?

A) Underfitting desde la epoch 15; conviene entrenar muchas más epochs.
B) Exploding gradients; conviene reducir el batch size a 1.
C) Learning rate demasiado pequeño; conviene multiplicarlo por 100.
D) Overfitting desde la epoch 15; conviene parar ahí (early stopping) o regularizar más.
E) Ruido en el conjunto de test; conviene ignorar la val loss.

10. ¿Qué hace RMSprop con el gradiente?

A) Lo suma con una fracción de gradientes anteriores, sin normalizar.
B) Lo recorta a un valor máximo para evitar gradientes explosivos.
C) Lo divide por la raíz de una media móvil de gradientes al cuadrado, adaptando el paso por parámetro.
D) Lo multiplica por la probabilidad de Dropout de cada capa.
E) Lo calcula siempre sobre una única muestra elegida al azar.

11. En PyTorch, con `nn.Dropout(p=0.5)` en modo train, ¿por qué factor se multiplican las activaciones que se mantienen?

A) Por 0.5, para reducir su magnitud a la mitad.
B) Por 2, para conservar el valor esperado de la salida.
C) Por 1: las activaciones restantes no se modifican.
D) Por 0.25, para compensar las dos mitades.
E) Por 4, para compensar la capa siguiente.

12. ¿Con qué conjunto se deben elegir los hiperparámetros (learning rate, nº de neuronas, etc.)?

A) Con el conjunto de test, porque es el más representativo.
B) Con el conjunto de entrenamiento, porque es el más grande.
C) Con la unión de train y test para tener más datos.
D) No hace falta elegirlos: PyTorch los ajusta solo.
E) Con el conjunto de validación; el de test se usa solo una vez al final.

13. Un modelo de regresión obtiene un R2 negativo en test. ¿Qué significa?

A) Que el modelo es perfecto, pero con las predicciones invertidas de signo.
B) Que el MSE es negativo y hay un error de cálculo.
C) Predice peor que un modelo trivial que siempre devolviera la media de los valores reales.
D) Que el modelo explica un 100 % de la varianza con signo contrario.
E) Que el modelo tiene overfitting, pero acierta la tendencia.

14. Una red tiene 5 capas Linear apiladas y ninguna función de activación entre ellas. ¿A qué equivale?

A) A una red profunda capaz de aproximar cualquier función continua.
B) A una red con 5 veces más capacidad que una sola capa.
C) A una red que solo puede resolver problemas de clasificación.
D) A una única transformación lineal (una sola capa Linear).
E) A una red con Dropout implícito en cada capa.

PARTE II - Análisis de código (preguntas 15 a 25)

Responde a las preguntas 15 a 25 usando el siguiente script de PyTorch. Se asume que el código se ejecuta sin errores salvo que una pregunta indique una modificación.

```
1  import numpy as np
2  import torch
3  from torch import nn
4  from torch.utils.data import TensorDataset, DataLoader
5  from sklearn.datasets import fetch_california_housing
6  from sklearn.preprocessing import StandardScaler
7
```

```
 8  data = fetch_california_housing()    # 20640 muestras, 8 variables
 9  X, y = data.data, data.target        # y: precio medio (continuo)
10
11  idx = np.random.permutation(20640)
12  tr, va = idx[:16000], idx[16000:]    # 4640 muestras de validación
13
14  scaler = StandardScaler().fit(X[tr])
15  X_train = torch.tensor(scaler.transform(X[tr]), dtype=torch.float)
16  X_val = torch.tensor(scaler.transform(X[va]), dtype=torch.float)
17  y_train = torch.tensor(y[tr], dtype=torch.float).unsqueeze(1)
18  y_val = torch.tensor(y[va], dtype=torch.float).unsqueeze(1)
19
20  net = nn.Sequential(
21      nn.Linear(8, 64),
22      nn.ReLU(),
23      nn.Linear(64, 32),
24      nn.ReLU(),
25      nn.Linear(32, 1),
26  )
27
28  criterion = nn.MSELoss()
29  optim = torch.optim.SGD(net.parameters(), lr=0.01, momentum=0.9)
30
31  train_loader = DataLoader(TensorDataset(X_train, y_train), batch_size=128, shuffle=True)
32  val_loader = DataLoader(TensorDataset(X_val, y_val), batch_size=128)
33
34  best_val, patience, wait = float("inf"), 5, 0
35  for epoch in range(100):
36      net.train()
37      for x, t in train_loader:
38          optim.zero_grad()
39          pred = net(x)
40          loss = criterion(pred, t)
41          loss.backward()
42          optim.step()
43
44      net.eval()
45      val_loss = 0.
46      with torch.no_grad():
47          for x, t in val_loader:
48              val_loss += criterion(net(x), t).item() * x.size(0)
49      val_loss /= len(X_val)
50
51      if val_loss < best_val:
52          best_val, wait = val_loss, 0
53      else:
54          wait += 1
55          if wait >= patience:
56              break
```

15. ¿Cuántos parámetros entrenables tiene `net` (líneas 20-26)?

A) 2689
B) 2592
C) 2657
D) 2720
E) 2561

16. Se cambia la línea 23 por `nn.Linear(64, 16)` y la línea 25 por `nn.Linear(16, 1)`. ¿Cuántos parámetros tiene ahora la red?

A) 1552
B) 1616
C) 1649
D) 2689
E) 1633

17. ¿Cuál es la forma de `pred` en una iteración de entrenamiento con batch completo (línea 39)?

A) (128,)
B) (16000, 1)
C) (128, 1)
D) (1, 128)
E) (128, 8)

18. El entrenamiento se detiene por early stopping tras completar 23 epochs. ¿Cuántas veces se ha ejecutado `optim.step()`?

A) 23
B) 125
C) 2875
D) 2300
E) 12500

19. El `val_loader` tiene 4640 muestras y `batch_size=128`. ¿Cuántos batches recorre y de qué tamaño?

A) 36 batches de 128 muestras; las 32 restantes se descartan.
B) 37 batches de 128 muestras cada uno.
C) 4640 batches de una muestra cada uno.
D) 36 batches: 35 de 128 y el último de 160.
E) 37 batches: 36 de 128 muestras y uno de 32.

20. Supón que la mejor `val_loss` se alcanza en la epoch 12 (contando desde 1) y nunca vuelve a mejorarse. ¿Al final de qué epoch se ejecuta el `break`?

A) 12
B) 17
C) 13
D) 16
E) 18

21. Se elimina `.unsqueeze(1)` de la línea 17, de modo que `y_train` tiene forma (N,) en lugar de (N, 1). ¿Qué ocurre?

A) Siempre lanza un error por formas incompatibles.
B) No hay error, pero el broadcasting compara cada predicción con todas las etiquetas y la loss es incorrecta (con aviso).
C) No cambia nada: PyTorch reordena las dimensiones automáticamente.
D) Solo se pierde la última muestra de cada batch.
E) La loss es correcta y el entrenamiento mejora al sobrar una dimensión.

22. ¿Por qué el scaler se ajusta solo con `X[tr]` (línea 14) y no con `X` completo?

A) Porque StandardScaler no puede ajustarse con más de 16000 filas.
B) Para que la validación quede sin escalar y sirva de control.
C) Porque así se evita normalizar la variable objetivo.
D) Para evitar fuga de información: la validación no debe influir en el preprocesado.
E) Para que el scaler aprenda también los pesos de la primera capa.

23. Se añade `nn.Sigmoid()` tras la última `nn.Linear`. El precio objetivo llega hasta aprox. 5. ¿Qué problema aparece?

A) Ninguno: Sigmoid es la activación estándar en regresión.
B) El MSE deja de poder calcularse con salidas positivas.
C) La red pasa a devolver probabilidades que suman 1.
D) La salida queda limitada al rango (0, 1), así que no puede predecir valores mayores que 1.
E) Los gradientes de la primera capa se hacen exactamente cero.

24. Se elimina `net.eval()` (línea 44). ¿Cambia el resultado de la validación?

A) No: la red no contiene Dropout ni BatchNorm, que son las capas que se comportan distinto.
B) Sí: se activan los gradientes y los pesos se modifican con los datos de validación.
C) Sí: la ReLU se comporta de forma distinta en entrenamiento y evaluación.
D) Sí: el optimizador pasa a actualizar con los datos de validación.
E) Sí: el código lanza un error al faltar el modo evaluación.

25. Se sustituye `nn.MSELoss()` (línea 28) por `nn.L1Loss()`. ¿Qué cambia?

A) Se minimiza la Cross-Entropy, adecuada para variables continuas.
B) Se minimiza la raíz del MSE, que penaliza más los errores grandes.
C) Se minimiza el error absoluto medio (MAE), menos sensible a valores atípicos que el MSE.
D) Se penaliza la norma L1 de los pesos como regularización.
E) El código lanza un error al no usar etiquetas enteras.

EXAMEN 3

Redes convolucionales (CNN) sobre MNIST
Nombre: _____ Nota: _____ / 25

PARTE I - Teoría (preguntas 1 a 14)

1. Una imagen de 32x32 pasa por `Conv2d` con kernel 5x5, stride 1 y sin padding. ¿Qué tamaño espacial tiene la salida?

A) 28x28
B) 32x32
C) 30x30
D) 27x27
E) 26x26

2. Una imagen de 28x28 pasa por `Conv2d` con kernel 3, stride 2 y padding 1. ¿Qué tamaño espacial tiene la salida?

A) 13x13
B) 15x15
C) 28x28
D) 27x27
E) 14x14

3. ¿Cuántos parámetros entrenables tiene `nn.Conv2d(3, 16, kernel_size=5)` con bias?

A) 1200
B) 256
C) 1232
D) 75
E) 1216

4. ¿Qué forma tiene el resultado de aplicar `nn.MaxPool2d(2)` a un tensor de forma (16, 24, 24) (canales, alto, ancho)?

A) (8, 24, 24)
B) (16, 24, 24)
C) (16, 12, 12)
D) (16, 6, 6)
E) (4, 12, 12)

5. ¿Qué ventaja principal tiene una capa convolucional frente a una capa Linear para procesar imágenes?

A) Evita la necesidad de usar funciones de activación.
B) Garantiza que la red no pueda hacer overfitting.
C) Procesa solo los píxeles centrales y descarta los bordes.
D) Comparte los mismos pesos por toda la imagen y usa conexiones locales, lo que reduce mucho los parámetros.
E) Convierte la imagen en un vector antes de procesarla.

6. Dos capas convolucionales 3x3 con stride 1 y sin pooling, seguidas. ¿Qué región de la entrada «ve» cada neurona de la segunda capa?

A) 3x3
B) 6x6
C) 5x5
D) 9x9
E) 4x4

7. ¿En qué consiste el data augmentation en clasificación de imágenes?

A) En generar variantes de las imágenes de entrenamiento (giros, recortes, brillo) conservando su etiqueta.
B) En añadir ruido aleatorio a las etiquetas para regularizar.
C) En aumentar la resolución del conjunto de test antes de evaluar.
D) En duplicar las capas de la red para ganar capacidad.
E) En entrenar con el conjunto de test para ampliar los datos.

8. En transfer learning con una CNN preentrenada y pocos datos propios, ¿qué estrategia es habitual?

A) Reinicializar todos los pesos y entrenar desde cero.
B) Congelar las capas convolucionales y entrenar solo un clasificador final nuevo.
C) Eliminar las capas convolucionales y usar solo capas Linear.
D) Entrenar únicamente con el conjunto de test.
E) Aumentar el learning rate a 10 para que la red se adapte rápido.

9. ¿Qué estadísticos usa una capa BatchNorm en modo evaluación (`model.eval()`)?

A) La media y la varianza del batch de evaluación actual.
B) Las medias y varianzas acumuladas (running statistics) durante el entrenamiento.
C) Media 0 y varianza 1 fijas, sin cálculo alguno.
D) Las de la primera muestra del conjunto de test.
E) Ninguno: BatchNorm se desactiva por completo.

10. Los logits de una muestra son (ln 2, 0, 0), con ln 2 aprox. 0.69. ¿Qué devuelve Softmax?

A) (0.69, 0.00, 0.00)
B) (0.33, 0.33, 0.33)
C) (0.40, 0.30, 0.30)
D) (0.50, 0.25, 0.25)
E) (0.67, 0.17, 0.17)

11. Una red recién inicializada reparte la probabilidad por igual entre 4 clases. ¿Qué loss (Cross-Entropy) esperas en promedio?

A) 0.25
B) 0.69
C) 2.00
D) 4.00
E) ln 4, aprox. 1.39

12. Una matriz de confusión binaria da TP = 40, FP = 10, FN = 20, TN = 30. ¿Cuánto vale el recall de la clase positiva?

A) 0.67
B) 0.80
C) 0.70
D) 0.33
E) 0.75

13. ¿Cuál de las siguientes medidas ayuda a mitigar el vanishing gradient en redes profundas?

A) Apilar más capas con sigmoide para compensar.
B) Inicializar todos los pesos a cero.
C) Reducir el learning rate a un valor cercano a cero.
D) Usar batches de una sola muestra.
E) Usar ReLU (u otras activaciones no saturantes) en las capas ocultas.

14. Un scheduler `StepLR` con lr inicial 0.1, `step_size=10` y `gamma=0.1` se actualiza una vez por epoch. ¿Qué lr se usa en la epoch 25 (contando desde 0)?

A) 0.1
B) 0.01
C) 0.0001
D) 0.025
E) 0.001

PARTE II - Análisis de código (preguntas 15 a 25)

Responde a las preguntas 15 a 25 usando el siguiente script de PyTorch. Se asume que el código se ejecuta sin errores salvo que una pregunta indique una modificación.

```
1  import torch
2  from torch import nn
3  from torchvision import datasets, transforms
4  from torch.utils.data import DataLoader
5
6  transform = transforms.ToTensor()
7  train_set = datasets.MNIST("data", train=True, download=True, transform=transform)
```

```
 8  train_loader = DataLoader(train_set, batch_size=64, shuffle=True)  # 60000 imágenes
 9
10  class CNN(nn.Module):
11      def __init__(self):
12          super().__init__()
13          self.conv1 = nn.Conv2d(1, 8, kernel_size=3)
14          self.pool = nn.MaxPool2d(2)
15          self.conv2 = nn.Conv2d(8, 16, kernel_size=3)
16          self.fc1 = nn.Linear(16 * 5 * 5, 64)
17          self.fc2 = nn.Linear(64, 10)
18
19      def forward(self, x):
20          x = self.pool(torch.relu(self.conv1(x)))
21          x = self.pool(torch.relu(self.conv2(x)))
22          x = x.flatten(start_dim=1)
23          x = torch.relu(self.fc1(x))
24          return self.fc2(x)
25
26  model = CNN()
27  criterion = nn.CrossEntropyLoss()
28  optim = torch.optim.Adam(model.parameters(), lr=1e-3)
29
30  for epoch in range(5):
31      for x, y in train_loader:
32          optim.zero_grad()
33          loss = criterion(model(x), y)
34          loss.backward()
35          optim.step()
```

15. ¿Cuántos parámetros entrenables tiene `CNN`?

A) 27482
B) 27546
C) 27562
D) 27650
E) 25664

16. ¿Cuántos parámetros tiene solo la capa `conv2` (línea 15)?

A) 1168
B) 1152
C) 1184
D) 2336
E) 144

17. Con un batch de 64 imágenes, ¿qué forma tiene la salida de `self.conv1(x)` (línea 20)?

A) (64, 8, 28, 28)
B) (64, 8, 13, 13)
C) (64, 1, 26, 26)
D) (64, 8, 26, 26)
E) (8, 26, 26)

18. Con un batch de 64 imágenes, ¿qué forma tiene `x` justo después de la línea 22 (`flatten`)?

A) (64, 400)
B) (64, 16, 5, 5)
C) (64, 25)
D) (25600,)
E) (64, 64)

19. ¿Cuántas veces se ejecuta `optim.step()` en todo el entrenamiento (5 epochs, 60000 imágenes, batch 64)?

A) 469
B) 4690
C) 937
D) 4685
E) 60000

20. ¿Qué tamaño tiene el último batch de cada epoch?

A) 64
B) 32
C) 16
D) 60
E) 0 (el último batch incompleto se descarta)

21. Se define `conv1` con `padding=1` y no se cambia nada más. ¿Qué ocurre al ejecutar?

A) Funciona igual, porque el padding no cambia el tamaño de los mapas.
B) Funciona, pero con 16 parámetros más en `conv1`.
C) Error en `conv2`, que no admite mapas de 28x28.
D) Error de dimensiones: `fc1` espera 400 entradas y recibe 576.
E) Funciona, y `fc1` pasa a tener 576 entradas de forma automática.

22. Se cambia `conv1` (línea 13) a `kernel_size=5` sin padding. ¿Qué ocurre?

A) No da error y el total sigue siendo 27562 parámetros.
B) Da error de dimensiones en `fc1`.
C) No da error y el total pasa a 27690 parámetros.
D) No da error y el total pasa a 27770 parámetros.
E) No da error y el total pasa a 27650 parámetros.

23. Se elimina la línea 22 (`x = x.flatten(start_dim=1)`). ¿Qué ocurre?

A) Funciona igual, porque Linear aplana la entrada automáticamente.
B) Funciona, pero la salida pasa a tener forma (64, 16, 10).
C) Error de producto matricial: `fc1` opera sobre la última dimensión, que vale 5, y espera 400.
D) Error porque `pool` no admite tensores de 4 dimensiones.
E) Funciona, pero solo usa el primer canal de cada imagen.

24. `self.pool` (línea 14) se llama dos veces en `forward`. ¿Afecta eso al número de parámetros?

A) Sí: ambos usos comparten pesos y el total baja en 1168 parámetros.
B) Sí: lanza error porque un módulo no puede llamarse dos veces.
C) Sí: cada uso añade 16 parámetros, uno por canal.
D) No: MaxPool2d no tiene parámetros entrenables, así que compartirlo no cambia nada.
E) Sí: el segundo uso reinicia los pesos del primero.

25. ¿Qué devuelve `transforms.ToTensor()` (línea 6) para una imagen de MNIST?

A) Un tensor de enteros (28, 28) con valores entre 0 y 255.
B) Un tensor float de forma (1, 28, 28) con valores escalados al rango [0, 1].
C) Un tensor de forma (784,) estandarizado a media 0.
D) Un tensor float de forma (28, 28, 1) con valores en [0, 255].
E) Un tensor de forma (1, 784) con valores en [-1, 1].

EXAMEN 4

Clasificación binaria con BCEWithLogits, scheduler y evaluación (Breast Cancer)
Nombre: _____ Nota: _____ / 25

PARTE I - Teoría (preguntas 1 a 14)

1. Un parámetro vale w = 0.5 y su gradiente es 0.2. Con SGD simple y lr = 0.1, ¿cuánto vale tras un paso?

A) 0.52
B) 0.30
C) 0.70
D) 0.48
E) 0.40

2. Según la implementación de PyTorch, SGD con momentum 0.9 y lr = 0.1 tiene velocidad previa v = 1.0. Llega un gradiente g = 0.5 y el peso vale w = 2.0. ¿Cuánto vale w tras el paso?

A) 1.95
B) 1.86
C) 1.91
D) 1.85
E) 2.14

3. Una neurona calcula y = ReLU(w*x + b) con x = 2, w = -1 y b = 0.5. ¿Cuánto vale dy/dw?

A) 2, porque dy/dw = x.
B) -1, porque dy/dw = w.
C) -3, porque dy/dw = w*x.
D) 0.5, porque dy/dw = b.
E) 0, porque z = -1.5 y la ReLU tiene derivada nula para entradas negativas.

4. ¿Qué técnica se usa habitualmente contra los exploding gradients?

A) Aumentar el learning rate para compensar.
B) Gradient clipping: limitar la norma del gradiente a un valor máximo.
C) Añadir más capas con activación sigmoide.
D) Quitar la normalización de las variables de entrada.
E) Aumentar el número de epochs hasta que se estabilice.

5. Un clasificador binario obtiene un AUC-ROC de 0.5. ¿Qué significa?

A) No discrimina mejor que el azar entre las dos clases.
B) Acierta exactamente la mitad de las muestras positivas.
C) Es un clasificador perfecto cuando el umbral vale 0.5.
D) Tiene 50 % de precision y 50 % de recall en cualquier umbral.
E) Siempre predice la clase mayoritaria.

6. ¿Para qué sirve `pos_weight` en `BCEWithLogitsLoss`?

A) Para fijar el umbral de decisión de la clase positiva en 0.5.
B) Para ponderar cada neurona de la capa oculta según su peso.
C) Para dar más peso a los errores sobre la clase positiva, útil con clases desbalanceadas.
D) Para normalizar los logits antes de aplicar la sigmoide.
E) Para elegir cuántas muestras positivas entran en cada batch.

7. En validación cruzada con k = 5, ¿qué se cumple?

A) Cada muestra se usa solo una vez, siempre para entrenar.
B) El 5 % de los datos se reserva como test final.
C) Cada muestra se usa exactamente una vez para validar y k-1 = 4 veces para entrenar.
D) Se entrenan 5 modelos con los mismos datos de validación.
E) Cada muestra se usa para validar en las 5 iteraciones.

8. En clasificación binaria se baja el umbral de decisión de 0.5 a 0.3. ¿Qué efecto esperas para la clase positiva?

A) Se predicen menos positivos: la precision sube y el recall baja.
B) El recall y la precision suben siempre a la vez.
C) Se predicen más positivos: el recall suele subir y la precision bajar.
D) No cambia nada: el umbral solo afecta a la loss de entrenamiento.
E) Los pesos de la red se reentrenan para ajustarse al nuevo umbral.

9. Una variable categórica «color» toma los valores {rojo, verde, azul}. ¿Qué codificación es preferible como entrada de la red?

A) Un entero (1, 2, 3), para que la red aprenda la relación entre colores.
B) Su media sobre el dataset, repetida en todas las filas.
C) Una columna con el número de letras de cada nombre.
D) One-hot (3 columnas binarias), para no introducir un orden artificial entre colores.
E) No se puede usar una variable categórica en una red.

10. Se multiplica por 4 el `batch_size`, manteniendo dataset y epochs. ¿Qué ocurre con el nº de actualizaciones de parámetros?

A) Se multiplica por 4, porque cada batch aporta más información.
B) No cambia, porque depende solo del número de epochs.
C) Se divide entre 16, al ser el efecto cuadrático.
D) Se anula, porque con batches grandes no hay backpropagation.
E) Se divide aproximadamente entre 4: hay menos actualizaciones, cada una con un gradiente menos ruidoso.

11. Una red recibe 6 variables y tiene dos capas ocultas de 4 neuronas y una salida de 1 neurona (todas con bias). ¿Cuántos parámetros entrenables tiene?

A) 44
B) 49
C) 57
D) 53
E) 64

12. Clasificación de 5 clases excluyentes con `CrossEntropyLoss`. ¿Cuántas neuronas de salida y qué activación?

A) 1 neurona con Sigmoid; la clase se obtiene redondeando.
B) 5 neuronas sin activación (logits): CrossEntropyLoss ya aplica log-softmax.
C) 5 neuronas con Sigmoid; se elige la de mayor valor sin más.
D) 4 neuronas, porque la quinta clase se deduce de las demás.
E) 5 neuronas con ReLU para evitar probabilidades negativas.

13. La train loss y la val loss son ambas altas, parecidas entre sí, y se estancan. ¿Qué cambio es más razonable?

A) Añadir más Dropout y más regularización L2.
B) Reducir el número de neuronas de la red.
C) Aumentar la capacidad (más neuronas o capas) o entrenar durante más tiempo.
D) Aplicar early stopping con paciencia 1.
E) Reducir el tamaño del conjunto de entrenamiento.

14. ¿Por qué se usa `shuffle=True` en el DataLoader de entrenamiento?

A) Para que los batches cambien en cada epoch y la red no aprenda el orden de los datos.
B) Para que el conjunto de validación se mezcle con el de entrenamiento.
C) Para duplicar muestras y aumentar el tamaño del dataset.
D) Porque sin shuffle PyTorch no calcula los gradientes.
E) Para que el último batch tenga siempre el tamaño completo.

PARTE II - Análisis de código (preguntas 15 a 25)

Responde a las preguntas 15 a 25 usando el siguiente script de PyTorch. Se asume que el código se ejecuta sin errores salvo que una pregunta indique una modificación.

```
1  import torch
2  from torch import nn
3  from torch.utils.data import TensorDataset, DataLoader
```

```
 4  from sklearn.datasets import load_breast_cancer
 5
 6  data = load_breast_cancer()    # 569 muestras, 30 variables, 2 clases
 7  X = torch.tensor(data.data, dtype=torch.float)
 8  y = torch.tensor(data.target, dtype=torch.float).unsqueeze(1)
 9
10  mean, std = X.mean(dim=0), X.std(dim=0)
11  X = (X - mean) / std
12
13  idx = torch.randperm(569)
14  X_train, y_train = X[idx[:400]], y[idx[:400]]
15  X_test, y_test = X[idx[400:]], y[idx[400:]]    # 169 muestras
16
17  model = nn.Sequential(
18      nn.Linear(30, 16),
19      nn.ReLU(),
20      nn.Linear(16, 1),
21  )
22
23  criterion = nn.BCEWithLogitsLoss()
24  optim = torch.optim.Adam(model.parameters(), lr=0.01, weight_decay=1e-4)
25  scheduler = torch.optim.lr_scheduler.StepLR(optim, step_size=10, gamma=0.5)
26
27  train_loader = DataLoader(TensorDataset(X_train, y_train), batch_size=64, shuffle=True)
28  test_loader = DataLoader(TensorDataset(X_test, y_test), batch_size=64)
29
30  for epoch in range(30):
31      for x, t in train_loader:
32          optim.zero_grad()
33          logits = model(x)
34          loss = criterion(logits, t)
35          loss.backward()
36          optim.step()
37      scheduler.step()
38
39  model.eval()
40  preds = []
41  with torch.no_grad():
42      for x, t in test_loader:
43          probs = torch.sigmoid(model(x))
44          preds.append((probs > 0.5).float())
45  preds = torch.cat(preds)
46  accuracy = (preds == y_test).float().mean().item()
```

15. ¿Cuántos parámetros entrenables tiene `model` (líneas 17-21)?

A) 496
B) 513
C) 529
D) 545
E) 481

16. Se cambia la línea 18 por `nn.Linear(30, 32)` y la línea 20 por `nn.Linear(32, 1)`. ¿Cuántos parámetros tiene ahora la red?

A) 1025
B) 992
C) 1057
D) 1089
E) 513

17. ¿Qué forma tiene `logits` (línea 33) en el último batch de cada epoch?

A) (64, 1)
B) (400, 1)
C) (16,)
D) (1, 16)
E) (16, 1)

18. ¿Cuántas veces se ejecuta `optim.step()` en todo el entrenamiento?

A) 210
B) 30
C) 180
D) 400
E) 12000

19. El `test_loader` tiene 169 muestras y `batch_size=64`. ¿Cuántos batches recorre y de qué tamaño?

A) 3 batches: 64, 64 y 41 muestras.
B) 2 batches: 64 y 105 muestras.
C) 3 batches de 56, 56 y 57 muestras.
D) 169 batches de una muestra cada uno.
E) 2 batches de 64 muestras; las 41 restantes se descartan.

20. El scheduler (línea 25) se actualiza una vez por epoch. ¿Qué learning rate se usa en la epoch 25 (contando desde 0)?

A) 0.01
B) 0.005
C) 0.00125
D) 0.0025
E) 0.0005

21. Se añade `nn.Sigmoid()` después de la línea 20 manteniendo `BCEWithLogitsLoss`. ¿Qué ocurre?

A) Lanza un error porque BCEWithLogitsLoss no admite salidas en [0,1].
B) Funciona sin errores, pero la sigmoide se aplica dos veces y el aprendizaje se degrada.
C) Es equivalente al código original: Sigmoid se ejecuta solo una vez.
D) La red aprende más rápido, porque la loss recibe probabilidades.
E) Solo falla en el último batch, que tiene 16 muestras.

22. ¿Qué problema metodológico tiene la normalización de las líneas 10-11?

A) Ninguno: así se asegura que train y test estén en la misma escala.
B) `X.std` no está definido para tensores de varias columnas.
C) Las variables quedan con media distinta de cero en train.
D) Media y desviación se calculan con todos los datos, incluido el test: hay fuga de información.
E) Se pierde la correspondencia entre `X` e `y` al normalizar.

23. En la línea 44, la condición `probs > 0.5` equivale a una condición sobre los logits. ¿Cuál?

A) logits > 0.5, porque la sigmoide no cambia el umbral.
B) logits > 1, pues sigmoid(1) = 0.5.
C) logits > 0, pues sigmoid(0) = 0.5.
D) logits < 0, ya que la clase positiva tiene logit negativo.
E) |logits| > 0.5, sin importar el signo.

24. Se elimina `.unsqueeze(1)` de la línea 8, así que `y` tiene forma (N,). ¿Qué ocurre?

A) Funciona igual: PyTorch añade la dimensión automáticamente.
B) Funciona, pero la loss se calcula con broadcasting sobre 64x64.
C) Solo falla en el último batch, de tamaño 16.
D) Funciona y la loss es idéntica, pero más rápida.
E) BCEWithLogitsLoss lanza un error: el target (64,) no coincide con los logits (64, 1).

25. ¿Qué efecto tiene `weight_decay=1e-4` en el optimizador (línea 24)?

A) Reduce el learning rate un 0.01 % en cada epoch.
B) Elimina las neuronas con peso menor que 1e-4.
C) Desactiva una fracción 1e-4 de neuronas en cada paso.
D) Normaliza la salida de cada capa a media 0.
E) Añade una penalización L2 que empuja los pesos hacia valores pequeños.

SOLUCIONARIO

Claves de respuesta de los cuatro exámenes.

Ex.	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25
1	D	E	A	C	D	B	C	E	E	B	A	A	D	D	E	B	B	D	C	C	B	A	C	E	A
2	D	A	E	A	A	E	B	B	D	C	B	E	C	D	A	E	C	C	E	B	B	D	D	A	C
3	A	E	E	C	D	C	A	B	B	D	E	A	E	E	C	A	D	A	B	B	D	C	C	D	B
4	D	B	E	B	A	C	C	C	D	E	D	B	C	A	B	A	E	A	A	D	B	D	C	E	E

Examen 1: respuestas explicadas

1. D) Softmax reparte la probabilidad en función de las diferencias entre logits. Si todos son iguales el reparto es uniforme: 1/4 = 0.25 por clase (y las probabilidades siempre suman 1).

2. E) Para x < 0 la ReLU vale 0 y su derivada también. Si una neurona cae siempre en esa zona, no recibe gradiente y sus pesos no se actualizan. No se satura por arriba (derivada 1 para x > 0).

3. A) Es un problema multietiqueta: cada salida es una probabilidad independiente (Sigmoid + BCE). Softmax obligaría a que las probabilidades sumaran 1, es decir, a que las clases fueran excluyentes.

4. C) Capa oculta: 10*6 + 6 = 66. Salida: 6*2 + 2 = 14. Total = 80. Quien responde 72 olvida los bias de las dos capas (60 + 12).

5. D) CE = -ln(p) con p la probabilidad de la clase verdadera: -ln(0.5) = ln 2 = 0.693. Solo vale 0 si p = 1.

6. B) Con un lr excesivo cada paso se pasa del mínimo, la loss oscila cada vez más y termina en inf/NaN. Un lr pequeño produce lo contrario: loss que baja muy despacio.

7. C) Batches por epoch: 1000/50 = 20. Actualizaciones = 20 * 4 epochs = 80 (una por batch).

8. E) BatchNorm normaliza por mini-batch (media 0, varianza 1) y luego aplica gamma y beta, que sí se aprenden. Estabiliza y acelera el entrenamiento.

9. E) Error alto en train y similar en validación = alto sesgo (underfitting). El overfitting se reconoce por una gran diferencia train/val, no por rendimiento bajo en ambos.

10. B) Adam mantiene medias móviles del gradiente (momentum) y del gradiente al cuadrado (como RMSprop), de modo que cada parámetro tiene su propio tamaño de paso.

11. A) La penalización |w| empuja a cero con fuerza constante, por lo que muchos pesos se anulan (sparsity). L2 los reduce de forma suave, pero casi nunca los hace exactamente cero.

12. A) Primero se divide y después se calculan media y desviación solo con train; esos valores se aplican a val y test. Si no, el modelo «ve» información del test (data leakage).

13. D) Más capacidad suele empeorar el overfitting. Dropout, early stopping, weight decay y más datos son las herramientas estándar contra él.

14. D) Acierta todas las A (95 %) y ninguna B (recall de B = 0). Con clases desbalanceadas hay que mirar recall, precision o F1 por clase, no solo accuracy.

15. E) Linear(13,16): 13*16+16 = 224. Linear(16,8): 16*8+8 = 136. Linear(8,3): 8*3+3 = 27. ReLU y Dropout no tienen parámetros. Total 387 (360 sería olvidar los bias).

16. B) 224 + (16*12+12 = 204) + (12*3+3 = 39) = 467. 436 es el resultado sin bias y 387 el de la red original.

17. B) 128 muestras / batch 32 = 4 batches por epoch (sin batch incompleto). 4 * 20 epochs = 80 llamadas.

18. D) Cada batch tiene 32 muestras y la última capa devuelve 3 logits (una por clase): (batch, clases) = (32, 3).

19. C) Por defecto (drop_last=False) el último batch es incompleto: 50 = 32 + 18.

20. C) En modo eval Dropout es la identidad (y BatchNorm usa sus estadísticos acumulados). `model.train()` la vuelve a activar. No toca pesos ni gradientes.

21. B) CrossEntropyLoss devuelve por defecto la media del batch. Multiplicar por el tamaño y dividir al final por el total da la media real, aunque el último batch sea más pequeño. No afecta al gradiente.

22. A) `no_grad` solo evita guardar el grafo. Sin él el resultado numérico es el mismo; los pesos no cambian porque nadie llama a `optim.step()` en validación.

23. C) `step()` usa los gradientes almacenados; si acaban de ponerse a cero, el paso no cambia nada (con Adam, 0/(0+eps) = 0). Sin `zero_grad()` en absoluto, los gradientes se acumularían.

24. E) `nn.CrossEntropyLoss` espera logits. Aplicar softmax antes comprime las entradas a [0,1] y reduce mucho los gradientes. Softmax no es idempotente: softmax(softmax(x)) != softmax(x).

25. A) val_acc = correct / 50, así que correct = 0.92 * 50 = 46.

Examen 2: respuestas explicadas

1. D) Errores: -1 y +3. Cuadrados: 1 y 9. Media: (1+9)/2 = 5. El MAE sería 2, la suma de cuadrados 10 y el RMSE 2.24.

2. A) Con activaciones saturables (sigmoid, tanh) y muchas capas, la regla de la cadena multiplica derivadas pequeñas. El gradiente llega casi nulo a las capas iniciales. Lo contrario es exploding gradients.

3. E) Con pesos iguales, cada neurona calcula lo mismo y recibe la misma actualización: nunca se diferencian. Por eso se inicializa al azar (Xavier/He).

4. A) tanh es una sigmoide reescalada: sale en (-1, 1) y tiene media cero, lo que suele favorecer el entrenamiento frente a la sigmoide en capas ocultas.

5. A) 4*5+5 = 25; 5*3+3 = 18; 3*1+1 = 4. Total 47. Sin bias serían 20+15+3 = 38.

6. E) 10 batches completos de 100 y uno final de 50 muestras: 11 batches.

7. B) Con variables en escalas muy diferentes la loss tiene valles alargados y el gradiente descendente zigzaguea. Estandarizar acelera y estabiliza la convergencia.

8. B) El gradiente de un mini-batch es una estimación ruidosa del gradiente total, pero mucho más barata, y permite varias actualizaciones por epoch.

9. D) Train mejora mientras validación empeora: la red memoriza. El punto óptimo es el mínimo de la val loss (epoch 15) y early stopping lo detecta.

10. C) Parámetros con gradientes grandes reciben pasos más pequeños y viceversa. La opción de «sumar fracción de gradientes previos» describe el momentum.

11. B) Es el «inverted dropout»: se escala por 1/(1-p) = 2 en entrenamiento para que la esperanza de la salida no cambie y en evaluación no haga falta reescalar.

12. E) Si se ajustan con test, este deja de ser una estimación honesta del rendimiento real (se «contamina»). Validación sirve para decidir; test, para informar.

13. C) R2 = 1 - SSE/SST. Si el error del modelo supera al de predecir siempre la media, SSE > SST y R2 < 0. El MSE nunca es negativo.

14. D) La composición de transformaciones lineales es lineal: W5(...(W1 x)) = W x. Sin no linealidades la profundidad no aporta capacidad.

15. A) 8*64+64 = 576; 64*32+32 = 2080; 32*1+1 = 33. Total 2689 (2592 sería sin bias).

16. E) 576 + (64*16+16 = 1040) + (16*1+1 = 17) = 1633.

17. C) Batch de 128 muestras y una única neurona de salida: (128, 1). Por eso la etiqueta lleva `unsqueeze(1)`.

18. C) 16000/128 = 125 batches por epoch (exactos). 125 * 23 = 2875.

19. E) 4640 = 36*128 + 32. Sin `drop_last`, el último batch (32 muestras) se mantiene.

20. B) Epoch 12: `wait` = 0. Epochs 13, 14, 15, 16, 17 hacen wait = 1, 2, 3, 4, 5. En la 17, `wait >= patience` (5) y se sale.

21. B) MSELoss emite un UserWarning y hace broadcasting de (128,1) con (128,) a una matriz (128,128). Se ejecuta, pero la loss deja de ser el MSE real y la red aprende mal.

22. D) Los estadísticos se calculan con train y se aplican (`transform`) a validación. Así la validación simula datos realmente nuevos.

23. D) En regresión con rango no acotado la salida debe ser lineal (sin activación). Una sigmoide impide predecir valores fuera de (0,1).

24. A) `eval()` solo cambia capas como Dropout y BatchNorm. En esta red (Linear + ReLU) no hay diferencia, aunque sigue siendo buena práctica llamarlo.

25. C) L1Loss = media de |pred - real| (MAE). Penaliza linealmente, así que los outliers pesan menos que con el cuadrado del MSE. No es la regularización L1.

Examen 3: respuestas explicadas

1. A) Tamaño = (N + 2P - K)/S + 1 = (32 - 5)/1 + 1 = 28.

2. E) floor((28 + 2*1 - 3)/2) + 1 = floor(27/2) + 1 = 13 + 1 = 14.

3. E) Cada filtro tiene 3*5*5 = 75 pesos; hay 16 filtros: 1200 pesos + 16 bias = 1216.

4. C) El pooling reduce alto y ancho a la mitad y no cambia el número de canales. No tiene parámetros.

5. D) Un mismo filtro se desliza por toda la imagen (pesos compartidos) y mira solo un entorno local. Detecta patrones sin importar su posición y necesita muchos menos parámetros.

6. C) La primera capa ve 3x3; la segunda combina 3x3 salidas vecinas, que cubren 3 + 2 = 5 píxeles por lado: campo receptivo 5x5.

7. A) Se transforman las imágenes de forma que la clase no cambie. Aumenta la variedad de los datos y reduce el overfitting.

8. B) Las capas convolucionales preentrenadas ya extraen características útiles. Con pocos datos se reutilizan congeladas y se entrena solo la cabeza.

9. B) Así la salida para una muestra no depende de las demás muestras del batch. En train se usan las estadísticas del batch y se van acumulando las globales.

10. D) exp(ln 2) = 2, exp(0) = 1. Suma = 4. Probabilidades: 2/4, 1/4, 1/4.

11. E) CE = -ln(1/4) = ln 4 = 1.386. Es una buena comprobación inicial: la loss de la primera epoch debería rondar ln(nº de clases).

12. A) Recall = TP/(TP+FN) = 40/60 = 0.67. 0.80 es la precision (40/50) y 0.70 la accuracy (70/100).

13. E) La derivada de la ReLU es 1 para x > 0, así que no encoge el gradiente. También ayudan una buena inicialización, BatchNorm y conexiones residuales.

14. E) Epochs 0-9: 0.1. Epochs 10-19: 0.01. Epochs 20-29: 0.001. La epoch 25 cae en el tercer tramo.

15. C) conv1: 1*8*3*3+8 = 80. conv2: 8*16*3*3+16 = 1168. fc1: 400*64+64 = 25664. fc2: 64*10+10 = 650. Total 27562. MaxPool y ReLU no tienen parámetros.

16. A) 16 filtros, cada uno con 8*3*3 = 72 pesos: 1152 pesos + 16 bias = 1168.

17. D) 8 filtros, y (28 - 3)/1 + 1 = 26 de alto y ancho. El pooling (a 13x13) se aplica después.

18. A) 28 -> conv 26 -> pool 13 -> conv 11 -> pool 5. Quedan 16 canales de 5x5: 16*25 = 400 valores por imagen.

19. B) 60000/64 = 937.5, así que hay 938 batches por epoch (937 completos y uno incompleto). 938 * 5 = 4690.

20. B) 60000 - 937*64 = 60000 - 59968 = 32 imágenes. `drop_last` es False por defecto, así que se mantiene.

21. D) Con padding=1: 28 -> 28 -> pool 14 -> conv 12 -> pool 6 -> 16*6*6 = 576. `fc1` está definida con 400 y PyTorch no la adapta.

22. C) 28 -> conv5 24 -> pool 12 -> conv3 10 -> pool 5: sigue habiendo 16*5*5 = 400. conv1 pasa de 80 a 1*8*25+8 = 208 parámetros: 27562 - 80 + 208 = 27690.

23. C) Linear actúa sobre la última dimensión. El tensor es (64, 16, 5, 5) y la última dimensión es 5, no 400: las formas no son multiplicables.

24. D) Un módulo puede reutilizarse todas las veces que se quiera. Si tuviera pesos, se compartirían; MaxPool2d no tiene, solo toma el máximo de cada ventana.

25. B) ToTensor pasa de (alto, ancho, canal) a (canal, alto, ancho) y divide entre 255. La estandarización (media/desviación) requiere `Normalize` aparte.

Examen 4: respuestas explicadas

1. D) w_nuevo = w - lr * grad = 0.5 - 0.1*0.2 = 0.48.

2. B) v = 0.9*1.0 + 0.5 = 1.4. w = 2.0 - 0.1*1.4 = 1.86. (1.95 ignora el momentum; 2.14 sumaría en vez de restar.)

3. E) Regla de la cadena: dy/dw = ReLU'(z) * x. Como z = -2 + 0.5 = -1.5 < 0, ReLU'(z) = 0 y el gradiente es 0 (neurona inactiva).

4. B) Con `torch.nn.utils.clip_grad_norm_` se reescala el gradiente cuando su norma supera un umbral, evitando pasos gigantes.

5. A) AUC = probabilidad de que una muestra positiva reciba más puntuación que una negativa. 0.5 es el azar; 1.0, discriminación perfecta.

6. C) Multiplica la contribución de las muestras positivas a la loss. Con pocos positivos, evita que la red los ignore.

7. C) Los datos se dividen en 5 particiones; en cada iteración una es validación y las otras 4, train. Todas hacen de validación exactamente una vez.

8. C) Un umbral más bajo clasifica como positivas más muestras: se capturan más positivos reales (recall sube) a costa de más falsos positivos (precision baja). El umbral se aplica solo en inferencia.

9. D) Un entero impone que azul > verde > rojo y que están equiespaciados. One-hot trata cada categoría de forma independiente (o un embedding si hay muchas).

10. E) Actualizaciones = (muestras / batch_size) * epochs. Con batch 4 veces mayor, 4 veces menos pasos.

11. D) 6*4+4 = 28; 4*4+4 = 20; 4*1+1 = 5. Total 53 (44 serían sin bias).

12. B) Una neurona por clase, con logits sin transformar. `CrossEntropyLoss` combina log-softmax + NLL. Para obtener la clase basta `argmax` sobre los logits.

13. C) Es underfitting (alto sesgo). Hace falta un modelo más expresivo o más entrenamiento. Regularizar más o reducir el modelo empeoraría el problema.

14. A) Barajar rompe posibles ordenaciones (por clase, por fecha...) y hace que los batches sean diferentes en cada epoch, lo que da gradientes más representativos. En validación/test no hace falta.

15. B) 30*16+16 = 496 y 16*1+1 = 17. Total 513 (496 es solo la primera capa).

16. A) 30*32+32 = 992 y 32*1+1 = 33. Total 1025.

17. E) 400 = 6*64 + 16, así que el séptimo batch tiene 16 muestras. Una neurona de salida da la forma (16, 1).

18. A) 7 batches por epoch (6 completos y uno de 16) * 30 epochs = 210.

19. A) 169 = 64 + 64 + 41. El último batch queda incompleto pero no se descarta.

20. D) Epochs 0-9: 0.01. Epochs 10-19: 0.005. Epochs 20-29: 0.0025 (se multiplica por gamma = 0.5 cada 10 epochs).

21. B) BCEWithLogitsLoss ya incluye la sigmoide (por estabilidad numérica). Con otra previa, la loss ve valores en (0,1) y sigmoid(.) los comprime a (0.5, 0.73): gradientes débiles y mal entrenamiento.

22. D) Los estadísticos deben calcularse solo con train y aplicarse a test. Si no, la evaluación es optimista (data leakage), aunque el efecto aquí sea pequeño.

23. C) La sigmoide es creciente y sigmoid(0) = 0.5, así que probs > 0.5 si y solo si logit > 0.

24. E) A diferencia de MSELoss (que avisa y hace broadcasting), BCEWithLogitsLoss exige que target e input tengan el mismo tamaño y lanza un ValueError.

25. E) weight_decay suma un término proporcional al peso en el gradiente, equivalente a regularización L2. Reduce el overfitting al penalizar pesos grandes.